Name: Vaishnavi PM
USN: 1RUA25BCA0114

Lab 1: Largest and smallest element in an array

main.c

Share Run

```c
#include <stdio.h>
int main() {
    int a[100], n, i;
    int larg, small;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    larg = a[0];
    small = a[0];

    for(i = 1; i < n; i++) {
        if(a[i] > larg) {
            larg = a[i];
        }
        if(a[i] < small) {
            small = a[i];
        }
    }

    printf("Largest = %d\n", larg);
    printf("Smallest = %d\n", small);

    return 0;
}
```

Output

Clear

```
Enter number of elements: 5
Enter 5 elements:
1
12
3
3
4
Largest = 12
Smallest = 1


=== Code Execution Successful ===5
```

Lab2: Factorial of an no using recursion pp

main.c

Share Run

```c
#include <stdio.h>

int fact(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n*fact(n-1);
}

int main() {
    int n, result;

    printf("Enter number: ");
    scanf("%d", &n);

    result = fact(n);

    printf("Factorial = %d",result);

    return 0;
}
```

Output

Clear

```
Enter number: 6
Factorial = 720

=== Code Execution Successful ===
```

Lab3: nth Fibonacci no using recursion

main.c

Share Run

```c
#include <stdio.h>

int fib(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n - 1) + fib(n - 2);
}

int main() {
    int n, result;

    printf("Enter the number: ");
    scanf("%d", &n);

    result = fib(n);

    printf("Fibonacci = %d",result);

    return 0;
}
```

Output

Clear

```
Enter the number: 6
Fibonacci = 8

=== Code Execution Successful ===
```

Lab4: Implementation of Singly Linked List

Programiz
C Online Compiler

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
int main() {
    struct Node *head,*first,*second;
    head=(struct Node*)malloc(sizeof(struct Node));
    first=(struct Node*)malloc(sizeof(struct Node));
    second=(struct Node*)malloc(sizeof(struct Node));
    head->data=10;
    first->data=20;
    second->data=30;
    head->next=first;
    first->next=second;
    second->next=NULL;

    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=5;
    newNode->next=head;
    head=newNode;

    struct Node *temp = head;
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }

    free(head);
    free(first);
    free(second);
    free(temp);
    return 0;
}
```

Output

```
5
10
20
30


=== Code Execution Successful ===
```

Lab5: implementation of singly linked list using insertion and deletion

Programiz C Online Compiler

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;


struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}


void insertAtBeginning(int data) {
    struct Node* newNode = createNode(data);
    newNode->next = head;
    head = newNode;
}


void insertAtEnd(int data) {
    struct Node* newNode = createNode(data);
    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node* temp = head;
```

Output

```
Linked List: 10 -> 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> NULL
Linked List: 20 -> NULL


=== Code Execution Successful ===
```

Programiz C Online Compiler

main.c

Share Run

```c
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}


void insertAtPosition(int data, int position) {
    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }
    if (position == 1) {
        insertAtBeginning(data);
        return;
    }
    struct Node* newNode = createNode(data);
    struct Node* temp = head;
    for (int i = 1; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}


void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
```

Output

```
Linked List: 10 -> 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> NULL
Linked List: 20 -> NULL


=== Code Execution Successful ===
```

Programiz C Online Compiler

main.c

Share Run

```c
void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }
    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }
    struct Node* temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}


void deleteByData(int data) {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }
    if (head->data == data) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL && temp->next->data != data) {
        temp = temp->next;
```

Output

```
Linked List: 10 -> 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> NULL
Linked List: 20 -> NULL


=== Code Execution Successful ===
```

Programiz C Online Compiler

main.c

Share Run

```c

void display() {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty!\n");
        return;
    }
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    insertAtBeginning(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtPosition(15, 2);
    display();

    deleteAtBeginning();
    display();

    deleteAtEnd();
    display();

    deleteByData(15);
    display();

    return 0;
}
```

Output

```
Linked List: 10 -> 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> 30 -> NULL
Linked List: 15 -> 20 -> NULL
Linked List: 20 -> NULL


=== Code Execution Successful ===
```

Lab6: implementation of doubly linked list using insertion and deletion

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
struct Node* head = NULL;

// Insert at beginning
void insertStart(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL)
        head->prev = newNode;

    head = newNode;
}
// Insert at end
void insertEnd(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* temp = head;

    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
```

Output

```
List: 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
    temp->next = newNode;
    newNode->prev = temp;
}
// Insert at position
void insertPosition(int data, int pos) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* temp = head;

    newNode->data = data;

    if (pos == 1) {
        insertStart(data);
        return;
    }
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Position not found\n");
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}
// Display list
void display() {
    struct Node* temp = head;

    printf("List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
```

Output

Clear

```
List: 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

Programiz
C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
    if (temp == NULL) {
        printf("Position not found\n");
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}
// Display list
void display() {
    struct Node* temp = head;

    printf("List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
// Main function
int main() {

    insertStart(10);
    insertStart(5);

    insertEnd(20);
    insertEnd(30);

    insertPosition(15, 3);

    display();

    return 0;
}
```

Output

Clear

```
List: 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

// Delete at beginning
void delete_begin()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }

    temp = head;
    head = head->next;

    if(head != NULL)
        head->prev = NULL;

    free(temp);
}

// Delete at end
void delete_end()
{
    struct node *temp;

    if(head == NULL)
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }

    temp = head;
    while(temp->next != NULL)
        temp = temp->next;

    if(temp->prev != NULL)
        temp->prev->next = NULL;
    else
        head = NULL;

    free(temp);
}

// Delete at position
void delete_middle(int pos)
{
    struct node *temp;
    int i;

    temp = head;

    for(i=1;i<pos;i++)
        temp = temp->next;

    temp->prev->next = temp->next;

    if(temp->next != NULL)
        temp->next->prev = temp->prev;

    free(temp);
}

// Display list
```

Output

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
// Display list
void display()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main()
{
    struct node *n1,*n2,*n3,*n4;
    // Creating nodes
    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

    // Assign data
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    n4->data = 40;

    // Linking nodes
    n1->prev = NULL;
    n1->next = n2;

    n2->prev = n1;
    n2->next = n3;

    n3->prev = n2;
    n3->next = n4;
```

Output

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
    // Assign data
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    n4->data = 40;

    // Linking nodes
    n1->prev = NULL;
    n1->next = n2;

    n2->prev = n1;
    n2->next = n3;

    n3->prev = n2;
    n3->next = n4;

    n4->prev = n3;
    n4->next = NULL;

    head = n1;

    printf("Original List:\n");
    display();

    delete_begin();
    printf("After delete begin:\n");
    display();

    delete_middle(2);
    printf("After delete middle:\n");
    display();

    delete_end();
    printf("After delete end:\n");
    display();

    return 0;
}
```

Output

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```

Lab7: implementation of Circular (Singly) linked list using insertion and deletion

main.c

```c
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

// Insert at beginning
void insert_begin(int val)
{
    struct node *newnode, *temp;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
    }
    else
    {
        temp = head;
        while(temp->next != head)
        {
            temp = temp->next;
        }
        newnode->next = head;
        temp->next = newnode;
        head = newnode;
    }
}

// Insert at end
void insert_end(int val)
```

Output

```
Circular Linked List:
5 -> 10 -> 15 -> 20 -> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
// Insert at end
void insert_end(int val)
{
    struct node *newnode, *temp;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
    }
    else
    {
        temp = head;
        while(temp->next != head)
        {
            temp = temp->next;
        }
        temp->next = newnode;
        newnode->next = head;
    }
}

// Insert at given position
void insert_pos(int val, int pos)
{
    struct node *newnode, *temp;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    temp = head;

    for(i=1; i<pos-1; i++)
    {
        temp = temp->next;
```

Output

```
Circular Linked List:
5 -> 10 -> 15 -> 20 -> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
        temp = temp->next;
    }
    newnode->next = temp->next;
    temp->next = newnode;
}
// Display
void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }
        temp = head;

    do
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while(temp != head);

    printf("HEAD\n");
}

int main()
{
    insert_begin(10);
    insert_begin(5);
    insert_end(20);
    insert_pos(15,3);

    printf("Circular Linked List:\n");
    display();

    return 0;
}
```

Output

Clear

```
Circular Linked List:
5 -> 10 -> 15 -> 20 -> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

// Delete at beginning
void delete_begin()
{
    struct node *temp,*last;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }

    temp = head;
    last = head;

    while(last->next != head)
        last = last->next;

    if(head->next == head)
        head = NULL;
    else
    {
        head = head->next;
        last->next = head;
    }

    free(temp);
}
```

Output

Clear

```
Original List:
10 -> 20 -> 30 -> 40 -> HEAD
After deleting beginning:
20 -> 30 -> 40 -> HEAD
After deleting end:
20 -> 30 -> HEAD
After deleting position 2:
20 -> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
// Delete at end
void delete_end()
{
    struct node *temp,*prev;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }

    temp = head;

    while(temp->next != head)
    {
        prev = temp;
        temp = temp->next;
    }

    if(temp == head)
        head = NULL;
    else
        prev->next = head;

    free(temp);
}

// Delete at given position
void delete_pos(int pos)
{
    struct node *temp,*nextnode;
    int i;

    temp = head;

    for(i=1;i<pos-1;i++)
        temp = temp->next;

```

Output

Clear

```
Original List:
10 -> 20 -> 30 -> 40 -> HEAD
After deleting beginning:
20 -> 30 -> 40 -> HEAD
After deleting end:
20 -> 30 -> HEAD
After deleting position 2:
20 -> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
    nextnode = temp->next;
    temp->next = nextnode->next;

    free(nextnode);
}

// Display
void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }

    temp = head;

    do
    {
        printf("%d -> ",temp->data);
        temp = temp->next;
    }while(temp != head);

    printf("HEAD\n");
}

int main()
{
    // Manually creating circular linked list
    struct node *n1,*n2,*n3,*n4;

    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

```

Output

Clear

```
Original List:
10 -> 20 -> 30 -> 40 -> HEAD
After deleting beginning:
20 -> 30 -> 40 -> HEAD
After deleting end:
20 -> 30 -> HEAD
After deleting position 2:
20 -> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
struct node *n1,*n2,*n3,*n4;

n1 = (struct node*)malloc(sizeof(struct node));
n2 = (struct node*)malloc(sizeof(struct node));
n3 = (struct node*)malloc(sizeof(struct node));
n4 = (struct node*)malloc(sizeof(struct node));

head = n1;

n1->data = 10;
n1->next = n2;

n2->data = 20;
n2->next = n3;

n3->data = 30;
n3->next = n4;

n4->data = 40;
n4->next = head;

printf("Original List:\n");
display();

delete_begin();
printf("After deleting beginning:\n");
display();

delete_end();
printf("After deleting end:\n");
display();

delete_pos(2);
printf("After deleting position 2:\n");
display();

return 0;
}
```

Output

Clear

```
Original List:
10 -> 20 -> 30 -> 40 -> HEAD
After deleting beginning:
20 -> 30 -> 40 -> HEAD
After deleting end:
20 -> 30 -> HEAD
After deleting position 2:
20 -> HEAD


=== Code Execution Successful ===
```

Lab8: implementation of Circular (doubly) linked list using insertion and deletion

main.c

Share Run

```c
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
    struct node *prev;
};
struct node *head = NULL;

// Insert at beginning
void insert_begin(int val)
{
    struct node *newnode,*temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
        newnode->prev = head;
    }
    else
    {
        temp = head->prev;
        newnode->next = head;
        newnode->prev = temp;

        temp->next = newnode;
        head->prev = newnode;

        head = newnode;
    }
}

// Insert at end
```

Output

Clear

```
Circular Doubly Linked List:
5 <-> 10 <-> 15 <-> 20 <-> 30 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
// Insert at end
void insert_end(int val)
{
    struct node *newnode,*temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
        newnode->prev = head;
    }
    else
    {
        temp = head->prev;
        temp->next = newnode;
        newnode->prev = temp;

        newnode->next = head;
        head->prev = newnode;
    }
}

// Insert at given position
void insert_pos(int val,int pos)
{
    struct node *newnode,*temp;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    temp = head;

    for(i=1;i<pos-1;i++)
        temp = temp->next;
```

Output

Clear

```
Circular Doubly Linked List:
5 <-> 10 <-> 15 <-> 20 <-> 30 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
    newnode->next = temp->next;
    newnode->prev = temp;

    temp->next->prev = newnode;
    temp->next = newnode;
}
// Display
void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
    temp = head;

    do
    {
        printf("%d <-> ",temp->data);
        temp = temp->next;
    }while(temp != head);

    printf("HEAD\n");
}
int main()
{
    insert_begin(10);
    insert_begin(5);
    insert_end(20);
    insert_end(30);
    insert_pos(15,3);

    printf("Circular Doubly Linked List:\n");
    display();
    return 0;
}
```

Output

Clear

```
Circular Doubly Linked List:
5 <-> 10 <-> 15 <-> 20 <-> 30 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
    struct node *prev;
};
struct node *head = NULL;

// Delete at beginning
void delete_begin()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
    temp = head;

    if(head->next == head)
    {
        head = NULL;
    }
    else
    {
        head = head->next;
        head->prev = temp->prev;
        temp->prev->next = head;
    }

    free(temp);
}
// Delete at end
void delete_end()
{
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
void delete_end()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
    temp = head->prev;

    if(temp == head)
    {
        head = NULL;
    }
    else
    {
        temp->prev->next = head;
        head->prev = temp->prev;
    }

    free(temp);
}
// Delete at given position
void delete_pos(int pos)
{
    struct node *temp;
    int i;

    temp = head;

    for(i = 1; i < pos; i++)
        temp = temp->next;

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;

    if(temp == head)
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
    if(temp == head)
        head = temp->next;

    free(temp);
}
// Display
void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
     temp = head;

    do
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while(temp != head);

    printf("HEAD\n");
}
int main()
{
    // Manually creating circular doubly linked list
    struct node *n1,*n2,*n3,*n4;

    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

    head = n1;

    n1->data = 10;
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```

main.c

```c
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

    head = n1;

    n1->data = 10;
    n1->next = n2;
    n1->prev = n4;

    n2->data = 20;
    n2->next = n3;
    n2->prev = n1;

    n3->data = 30;
    n3->next = n4;
    n3->prev = n2;

    n4->data = 40;
    n4->next = head;
    n4->prev = n3;

    printf("Original List:\n");
    display();

    delete_begin();
    printf("After deleting beginning:\n");
    display();

    delete_end();
    printf("After deleting end:\n");
    display();

    delete_pos(2);
    printf("After deleting position 2:\n");
    display();

    return 0;
}
```

Output

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```

GitHub repository link:
https://github.com/vaishnavipmbca25-sketch/DSA-1.git